

# Learning Management System

31.10.2026

**CS4433 LMS Project**

Mzinyathi Ndaba

# Table of Contents

**Table of Contents** ..... **1**
**Abstract** ..... **3**
**Introduction** ..... **4**
**Project Management Plan** ..... **4**
**Requirements Engineering** ..... **9**
Stakeholder Analysis and Conflict Resolution ..... 9
Ambiguities and conflicts ..... 9
Software Requirements Specification ..... 10
FUNCTIONAL REQUIREMENTS ..... 10
NON-FUNCTIONAL REQUIREMENTS ..... 12
USE CASE DIAGRAM ..... 13
**System Design Document** ..... **20**
Design assumptions ..... 20
1. Architecture ..... 20
Layered Architecture ..... 20
2. Class Diagram ..... 22
2 SOLID ..... 22
3. Behavioural Models ..... 24
Submission life cycle ..... 24
Activity Diagram with swimlanes ..... 25
Sequence Diagram: Submit assignment ..... 26
Sequence Diagram: Moderate and release marks ..... 27
4. Database Design ..... 28
Normalisation ..... 29
4 Data dictionary ..... 29
SQL schema ..... 30
Mapping tables ..... 31
**Phase 5: Verification and Validation** ..... **33**
5. Test strategy and test plan ..... 33
6 Implementation ..... 34
**Phase 7: Deployment** ..... **35**
7 Installation guide ..... 35
User manuals ..... 35

**Phase 8: Maintenance: Change Request CR1 ..... 37**
Classification and justification ..... 37
Impact analysis ..... 37
Regression testing ..... 38
**Phase 9: Professional, Legal and Ethical Issues ..... 39**
Data protection personal information ..... 39
Professional and ethical responsibilities of the developer ..... 39
**References ..... 39**

# Abstract

The proposed project is based on a Learning Management System of a school. The system is designed for four main users: administrators, lecturers, heads of departments, and students, with each user having access to functions relevant to their role. C++ will be used for the system, and SQLite for the database. The project covers the requirements engineering, design, testing, and deployment. The proposed LMS aims to make school affairs management more efficient, organised, and accessible while reducing the challenges of manual management of student information. The project also considers the feasibility of developing the system.

# Introduction

The project is about designing and developing a Learning Management System (LMS) for a secondary school. The system's goal is to offer a platform for managing courses, assignments, submissions, marks and feedback in an organised way. The system is designed to facilitate the management of learning and teaching for staff and students and to ensure accurate checking and recording of marks before they are released. The system follows a waterfall with V-Model verification and validation software development model.

The LMS will have four main actors: students, lecturers, heads of department (HoDs), and the system administrator. Each user will log into the system using a username and password and will only have access to the functions associated with their role. The administrator will manage user accounts, departments and courses, assign lecturers to courses, and enroll students. Lecturers will manage assignments, set deadlines and marks, grant extensions where necessary, mark submissions and provide written feedback. HoDs will review and moderate marks within their departments before approving and releasing them to students. Students will be able to view their enrolled courses and assignments, submit and resubmit work within the allowed period, and view marks and feedback once these have been released.

Submissions will move through defined stages such as submitted, late, marked, moderated and released. The system will also keep a record of changes made to marks, including who made the change, when it was made and the reason for the change.

The system will also address different requirements raised by the school's stakeholders. For example, late submissions will be handled according to the school's agreed late-submission policy rather than being automatically rejected.

The LMS will be developed as a C++ application. It will operate over the school's local network and must be able to offer offline access. Since the school has relatively old computers and up to 800 students may use the system, the system should be designed to accommodate the computers and should not crash while at its full capacity. Passwords and login information shall be stored in a database.

# Project Management Plan

## Feasibility Study

The study considers the technical, economic, operational, and schedule feasibility of the proposed system. For this Learning Management System (LMS), C++ shall be utilised as the main programming language. C++ provides efficient execution and can support low response times, making it suitable for a system where timely processing of user requests is required. A secure and reliable database is required to store user authentication and academic information. This includes student identification information, securely hashed passwords, and information required for password recovery. The database must also remain reliable under the expected system load, which is approximately 800 students. Student marks

and other academic records shall also be stored in the database. SQLite shall be used for database implementation due to its lightweight nature and ease of integration with C++. Thus, the system is technically feasible.

Each user will have access only to the functions permitted by their role. The administrator will manage user accounts, departments, and courses; assign lecturers to courses; and enroll students. Lecturers will create assignments, set deadlines and maximum marks, grant individual extensions, mark submissions, and provide feedback. HoDs will review and moderate marks before they are released, while students will view their courses and assignments, submit and resubmit work where permitted, and view released marks and feedback. The system must also accommodate the different stakeholder requirements identified during the initial interviews. For example, students must not see marks before they have been checked by the Head of Department. The principal has indicated that the system should maintain simplicity. Since the system will operate as a console or simple desktop application, it can be used within the school's local network without requiring an internet connection. Based on these considerations, the proposed system is operationally feasible.

Because the development may utilise current school and development resources, the suggested LMS is economically feasible. The system will run on an existing school computer or local network, and the project does not call for the creation of a web or mobile application. As a result, less infrastructure is required.

The system shall offer access to private details such as marks and login details to authorised personnel. Only lecturers and HoDs shall access students' marks, and each student shall only access their individual marks. Each user shall only have access to the system and functions assigned to their roles after logging into the system using their own private login details. The system will be secure hence legally feasible. The schedule of the proposed system has been partitioned into 7 weeks, and a hybrid of waterfall and v-model shall be utilised as the system's development model which shall help verify that the system aligns with all the requirements at all phases of its development. Although the schedule is quite demanding, the defined structure provides explicit milestones and avoids implementation from beginning which would significantly increase chances of system failure and program stagnation, the system's schedule is feasible.

The study suggests that the proposed LMS is feasible for the conditions defined. The required technologies are available and applicable for a local C++ application, the system can meet the operational requirements of the four user groups, and the use of existing infrastructure avoids additional costs. In addition, role-based access, secure password storage, and proper protection of student information can also meet legal and security requirements. The main feasibility challenges are the system's performance with the student population (800), the reliability of the database and the limited seven-week development schedule. These issues should therefore be monitored throughout development through testing and risk management. Therefore, it is recommended that development proceeds to the requirements engineering phase, subject to the feasibility assumptions and constraints identified above.

## Work Breakdown

The Learning Management System shall be completed within 7 weeks, and work shall be broken as follows:

**Week 1**: Project initiation and Feasibility study. This stage assesses the objectives and scope of the project and whether it is feasible to implement the project with the resources that will be available.

**Week 2**: Requirements engineering conducted and requirements correctly classified into categories: Functional and non-functional. The four actors' requirements shall be analysed.

**Week 3**: System design begins, this includes the architecture and database structure. The design ensures that all the four actors' requirements are met. User interfaces and the system's functionality is established. This shall be used to guide the subsequent implementation stage.

**Week 4**: Guided by the already complete stage of design, the implementation of the system shall take place. All user functions are addressed. The system will be implemented in the programming language C++ and the database in SQLite.
**Week 5**: The system shall be tested to ensure that all stakeholder requirements are satisfied. Unit, integration, system and acceptance testing take place.

**Week 6**: The system shall be released and fully tested. This is the final stage of system verification. Packaging, installation guide, and user manual are submitted.

| Activity | Dependencies | Milestones | W 1 | W 2 | W 3 | W 4 | W 5 | W 6 |
|---|---|---|---|---|---|---|---|---|
| Feasibility study | None | Feasibility approval | ■ | | | | | |
| Requirements engineering | Feasibility Study | Requirements specification completion | | ■ | | | | |
| System and database design | Requirements Engineering | Completion of design | | | ■ | | | |
| System development | System and database design | Core development completion | | | | ■ | ■ | |
| System validation | System development | Successful testing | | | | | ■ | |
| Deployment | System development and system validation | LMS deployed | | | | | | ■ |

## Risk Management

| Phase | Risk | Mitigation | Ownership | Likelihood | Impact |
|---|---|---|---|---|---|
| Week 2 | Conflicting requirements from stakeholders | All requirement s should be clarified before user implementation | Actors | Moderate | One of the actors not satisfied with the project |
| Week 4 | Data from database is lost | Regular database backups | Developer | Moderate | System functionality is compromise d |
| Week 1-6 | Insufficient time to complete the system | Only work on core features and manage time efficiently | Project developer | High | Incomplete project |
| Week 1-6 | Excessive workload | Divide into manageable tasks | Project developer | Moderate | System is left incomplete |
| Week 6 | Unauthorised access to some of the system's functions | Implement authentication and only role-based access | Project developer | Moderate | Security is compromise d |
| Week 6 | User struggle to utilise the system due to complexity | Design simple and user-friendly interface | Project developer | Moderate | System does not satisfy stakeholders requirements. |

# Requirements Engineering

## Stakeholder Analysis and Conflict Resolution

The learning management system has 7 stakeholders:

1.Lecturers: To manage assessments, release marks as soon as grading concludes and offer deadline extensions to students when needed.

2.Heads of Departments: To approve marks and monitor student performance through the learning management system and late submissions should be allowed for not more than 3 days and should be penalised accordingly.

3.Students: To be able to make changes to assignments before the deadline and the feedback should have comments not just marks and work should not be rejected if submitted after the deadline.

4.Principal: To ensure that the system only allows marks to be released after approval from the HoD (academic integrity is upheld) and the system should not require IT expertise from the users.

5.IT officer: The system should be compatible with the current school computers and the LMS should offer offline access. The system should respond quickly and be functional even at full student capacity

6.System Developer: To build a system that is secure and meets the user requirements.

7.System Administrator: Manages user accounts, departments, courses, lecturer assignments and student enrolments.

### Ambiguities and conflicts

| **CONFLICT** | **SOLUTION** |
|---|---|
| The lecturer wants to release marks as soon as marking is complete while the principal and HoD prefer that marks are released only after approval from the HoD | Marks will remain hidden from students until the HoD has moderated and approved them. Lecturers can still view and edit marks before moderation, subject to the system's rules. |
| The HoD requires a penalty of 10% per day for up to three days, while students oppose automatic rejection of late work. | Submissions will be accepted up to three days after the deadline, subject to a penalty of 10% of the assignment's maximum mark per late day.<br>Submissions after three days receive zero. |
| The principal wants an easy to use system, while the IT Officer requires protected passwords and restricted access. | The system will provide straightforward menus and role specific functions while enforcing authentication and authorisation in the background. |

| | |
|---|---|
| Lecturers want to grant extensions, but the rules do not specify how extensions affect late penalties. | Lecturers may grant individual extensions. The new deadline will be used to determine whether a submission is late. |

The parties will be given the draft resolutions to consider. The IT Officer will verify security and performance requirements, the HoD will confirm late-penalty regulations, and the Principal and HoD will accept the mark moderation and release policy. Lecturers and students will go over the prerequisites for submission, resubmission, extension, and feedback. The requirements will serve as the foundation for system design once they are approved.

## Software Requirements Specification

This Software Requirements Specification (SRS) is structured with reference to ISO/IEC/IEEE 29148, which provides guidance on requirements engineering and the documentation of system requirements. The specification identifies functional and non-functional requirements. Each requirement has a unique identifier, a MoSCoW priority, its source and a verifiable acceptance criterion.

**DEFINITIONS**

**Functional requirement (FR)**: A service that the system must provide.

**Non-functional requirement (NFR)**: A measurable constraint on the system.

**MoSCoW prioritisation**: Must have, Should have, Could have and Won't have in the current release.

**Source**: The stakeholder's statement.

### FUNCTIONAL REQUIREMENTS

| ID | Requirement | Priority | Source | Verification |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| FR-01 | The system shall authenticate every user by username and password before granting access to any function, and shall restrict the visible menu and available actions to that user's role | MUST | IT Officer | A login with valid credentials succeeds and shows only that role's menu, else login rejected. |
| FR-02 | The administrator shall manage users, courses and enrolments | MUST | Administrator role | A newly created account can immediately log in with its assigned role |
| FR-03 | The Administrator shall be able to assign one or more lecturers to a course. | MUST | Administrator role | Only a user with the Lecturer role can be assigned to a course |
| FR-04 | The Administrator shall be able to enrol a student in a course. | MUST | Administrator | An enrolled student sees the course in their course list |
| FR-05 | A Lecturer shall be able to create an assignment for a course they are assigned to specify a deadline and a maximum mark. | MUST | Lecturer role | Assignment will only be created when all details are specified |
| FR-06 | A Lecturer shall be able to grant an individual student an extended deadline for a specific assignment. | MUST | Lecturer role | The will only be able to submit post the deadline after an extension has been granted |
| FR-07 | A Student shall be able to submit work for an assignment they are enrolled in, at any time before the deadline and 3 days past it. | MUST | HoD and Student Representative | The assignment rules will apply based on the classification. The old submission |

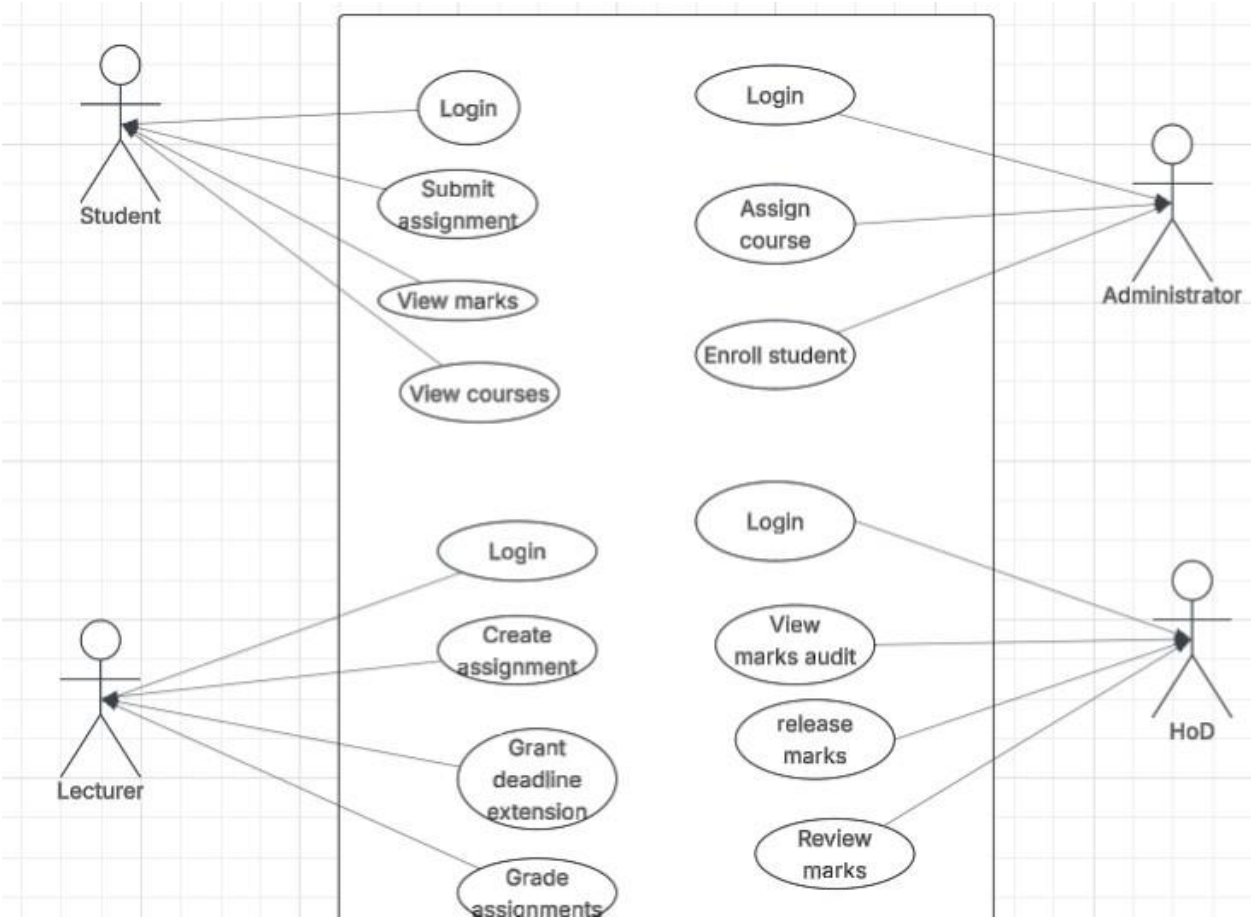
| | | | | |
|---|---|---|---|---|
| FR-08 | A Lecturer shall be able to record a mark and mandatory written feedback for a submission, and the mark shall not be saved without a filled feedback entry. | MUST | Student Representative | Marks will not be released when there is no comment alongside. |
| FR-09 | A Head of Department shall be able to view every mark and its feedback for every course in their department before it is released to students. | MUST | Head of Department and Principal | Only the head of department shall have the authority to release marks |
| FR-10 | A Head of Department shall be able to release an approved mark, after which it becomes visible to the student's owning. | MUST | Head of Department | After marks are released, a student will only see the mark they got, not those of other students. |
| FR-11 | The system shall automatically classify each submission and compute the applicable mark penalty using the rule of deducting 10% each day for three days. | MUST | System Requirement | Every submission will be classified as on time or late. |
| FR-12 | A Student shall be able to view a list of their enrolled courses and, for each, the assignments and their deadlines. | MUST | Student representative | A student will only have access to courses they have been enrolled into by the administrator. |

## NON-FUNCTIONAL REQUIREMENTS

| ID | Requirements | Priority | Category | Verification |
|---|---|---|---|---|
| NFR-01 | The system should be fully usable with no internet connectivity. | Must | Availability | All the system operations will be availed offline. |

| | | | | |
|---|---|---|---|---|
| NFR-02 | All users should be able to use the system with minimal training. | Must | Usability | Users with no prior experience of the system should be able to use it without struggle. |
| NFR-03 | The system should remain functional at maximum capacity of 800 students | Must | Reliability | The system will not crash when all users are active. |
| NFR-04 | Every mark change should be traceable | Must | Authenticity | The HoD should be able to see all alterations made to marks. |
| NFR-05 | The system should respond respond within at most 5 seconds | Should | Performance | The system will not lag when used. |
| NFR-06 | The user passwords should be well secured | Must | Reliability | Login will only be through passwords and password recoveries will be authorised by the administrator after verifying the user's identity. |

USE CASE DIAGRAM




**Use Case: Submit Assignment**

Primary Actor: Student

Description

This use case describes how a student submits an assignment through the LMS. The system verifies the student's eligibility, checks the submission details, determines whether the submission is late, applies the relevant penalty where necessary, and stores the submission for subsequent marking.

Preconditions

1. The student has a registered account and is logged into the system.
2. The student is enrolled in the course to which the assignment belongs.
3. The assignment exists and has been made available to the student.
4. The system and its database are operational.

Main Success Flow

1. The student selects an enrolled course.

2. The system displays the assignments available for that course.
3. The student selects the assignment to submit.
4. The system displays the assignment instructions, deadline, maximum mark and any applicable extension.
5. The student selects the work to submit and confirms the submission.
6. The system validates the student's enrollment, assignment availability and submission details.
7. The system determines whether the submission falls within the permitted submission period.
8. The system calculates any applicable late penalty.
9. The system stores the submission, submission timestamp and assessment information in the database.

Alternative Flows

At 5, if resubmission is permitted, the student submits a revised version. The system validates and stores the new version while retaining the submission history.

Submissions are late by up to three days. At 8, the system deducts 10% of the assignment's maximum mark for each day late, up to three days.

Exception Flows

If the submission is empty or fails the system's defined validation rules, the system displays an appropriate message and requests a valid submission.

If the system cannot store the submission, it informs the student that the submission was unsuccessful and does not display a successful confirmation.

If the same submission request is processed more than once, the system prevents unintended duplicate records.

Postconditions

1. The submission is associated with the correct student and assignment.
2. The submission timestamp and status are recorded.
3. Any applicable late penalty is used.
4. The student receives confirmation of the recorded submission. Business Rules

1. Only authenticated students enrolled in the relevant course may submit its assignments.
2. A student may resubmit before the applicable deadline, subject to the assignment's submission rules.
3. An approved extension changes the applicable deadline for the student concerned.
4. Late submissions lose 10% of the available mark for each day late, for up to 3 days.

**Mark Submission**

Primary Actor: Lecturer

Description

This use case describes how a lecturer evaluates a student's submitted assignment, records the mark and provides written feedback. The system validates the mark, applies any applicable late penalty, saves the assessment and maintains an audit trail of mark changes.

Preconditions

1. The lecturer is authenticated and authorised to access the relevant course.
2. The lecturer is assigned to the course containing the assignment.
3. The assignment and the student's submission exist in the system.
4. The submission is available for marking and has not already been finalised in a way that prevents further marking.

Main Success Flow

1. The lecturer selects a submission to mark.
2. The system displays the submitted work, assignment instructions, maximum mark and submission status.
3. The lecturer evaluates the work and enters a mark.
4. The lecturer provides written feedback explaining the assessment.
5. The system applies any applicable late submission penalty.
6. The system stores the resulting mark, feedback, lecturer identity and assessment timestamp.
7. The system updates the submission status to Marked

Alternative Flows

The lecturer corrects a mark. If the lecturer identifies an error before moderation and release, the lecturer enters a corrected mark and provides the required reason. The system saves the change and records it in the audit trail.

The lecturer saves a mark for a late submission. The system calculates the penalty according to the recorded submission time and applicable deadline before storing the resulting mark.
The lecturer reviews another submission. After completing the current assessment, the lecturer selects another submission and repeats the marking process.

The lecturer returns to the submission list. The lecturer exits the marking screen without changing the current assessment. The system returns to the submission list.

Exception Flows

If the entered mark is negative or exceeds the assignment's maximum mark, the system rejects it and requests a valid value.

If written feedback is required but has not been provided, the system requests the lecturer to complete it before saving the assessment.

Postconditions

1. The mark and written feedback are stored in the database.
2. The assessment is associated with the correct student, assignment and lecturer.
3. Any applicable late penalty is reflected in the resulting mark.
4. The submission status is updated to Marked.
5. The mark remains unavailable to the student until it has been moderated and released.

Business Rules

1. Only the lecturer assigned to the relevant course may mark its submissions.
2. A mark must be within the range from zero to the assignment's maximum mark.
3. Written feedback must accompany the assessment.
4. Late penalties must follow the approved policy.
5. Every change to an existing mark must be recorded, including who made the change, when it occurred and why it was made.
6. Students must not view marks before approval by the Head of Department.

**Review and Release Marks**

Primary Actor: Head of Department (HoD)

Description

This use case describes how the Head of Department reviews marks submitted by lecturers, checks the accuracy and fairness of assessments, records moderation decisions and releases approved marks to students. It ensures that students only gain access to marks after the departmental review.

Preconditions

1. The Head of Department is authenticated and authorised to perform moderation.
2. The Head of Department is associated with the relevant department.
3. The lecturer has completed marking for the submissions being reviewed.
4. The marks and feedback are stored in the system.

Main Success Flow

1. The Head of Department selects the departmental moderation function.
2. The system displays the courses and assessments within the department.
3. The Head of Department selects an assessment to review.
4. The system displays the recorded marks, feedback and relevant submission information.
5. The Head of Department reviews the marks for accuracy and consistency.
6. If the marks are satisfactory, the Head of Department approves the assessment.
7. The Head of Department selects the approved assessment for release.
8. The system changes the relevant assessment status to Released.
9. The system makes the approved marks and feedback available to the relevant students.

Alternative Flows

At 5, if an error is identified, the Head of Department records the issue and returns the assessment to the lecturer for correction. The marks cannot be released until the required correction and moderation have been completed.

The Head of Department exits without releasing the marks. The assessment remains moderated or pending release and the marks remain hidden from students.

Exception Flows

If required marks or feedback are missing, the system prevents the affected assessment from being approved for release.

If the Head of Department attempts to release marks before moderation has been completed, the system rejects the operation.
If a lecturer changes a mark while moderation is in progress, the system detects the change and requires the affected mark to be reviewed again before release.

If the moderation decision or release operation cannot be stored, the system reports the failure and retains the previous valid state.

Postconditions

1. The system records the Head of Department responsible for the decision and the relevant timestamp.
2. Approved marks are marked as Released.
3. Released marks and feedback become accessible to the students.
4. The audit trail maintains the history of relevant assessment changes and release actions.

Business Rules

1. Only an authorised Head of Department may moderate and release marks for the department concerned.
2. Every mark must be reviewed and approved before it is released to students.
3. Students must not access unmoderated or unreleased marks.
4. Marks that fail moderation must be corrected and reviewed before release.
5. Changes to marks must be recorded in the audit trail with the responsible user, timestamp and reason where required.
6. The system must maintain the confidentiality of academic records.

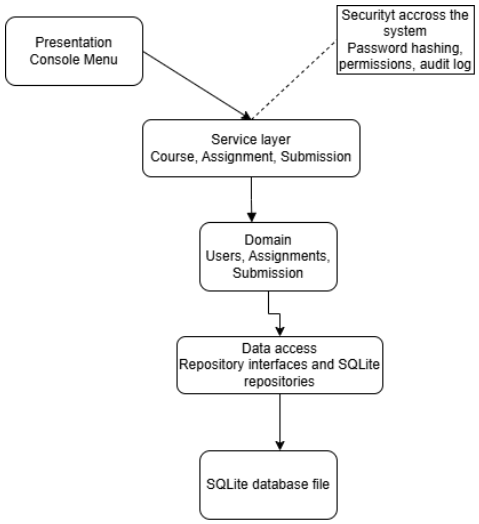
# System Design Document

## Design assumptions

| Conflict | Resolution used in this design |
|---|---|
| Principal and HoD want to hide marks until approved while the lecturer wants to release as soon as marking is completed | Marks are never visible to students until the HoD releases them. Lecturers see their marks at once, and the HoD moderates in bulk. |
| Lecturer wants extensions while fixed HoD penalty rule | An extension gives one student a personal effective deadline for one assignment. The penalty is then measured from that deadline. |
| Late-penalty wording | For 1–3 days late the penalty is 10% × maxMark × daysLate, taken off the awarded mark. For more than 3 days late the final mark is 0. |
| “Late work not rejected” vs “zero after 3 days” | Late work is always accepted and recorded as late, so it can still get feedback, but the penalty gives it 0 after day 3. |
| Resubmission | A student may resubmit only before the effective deadline. Every version is kept. The latest version is the one marked. |
| Offline access | Single-process C++17 app with embedded SQLite, indexed lookups, and no network dependency. |

## 1. Architecture

### Layered Architecture




Each layer communicates with the layer directly below it. The Presentation layer does not run SQL queries. The Domain layer is separate from both the user interface and SQLite, and accesses data through repository interfaces.

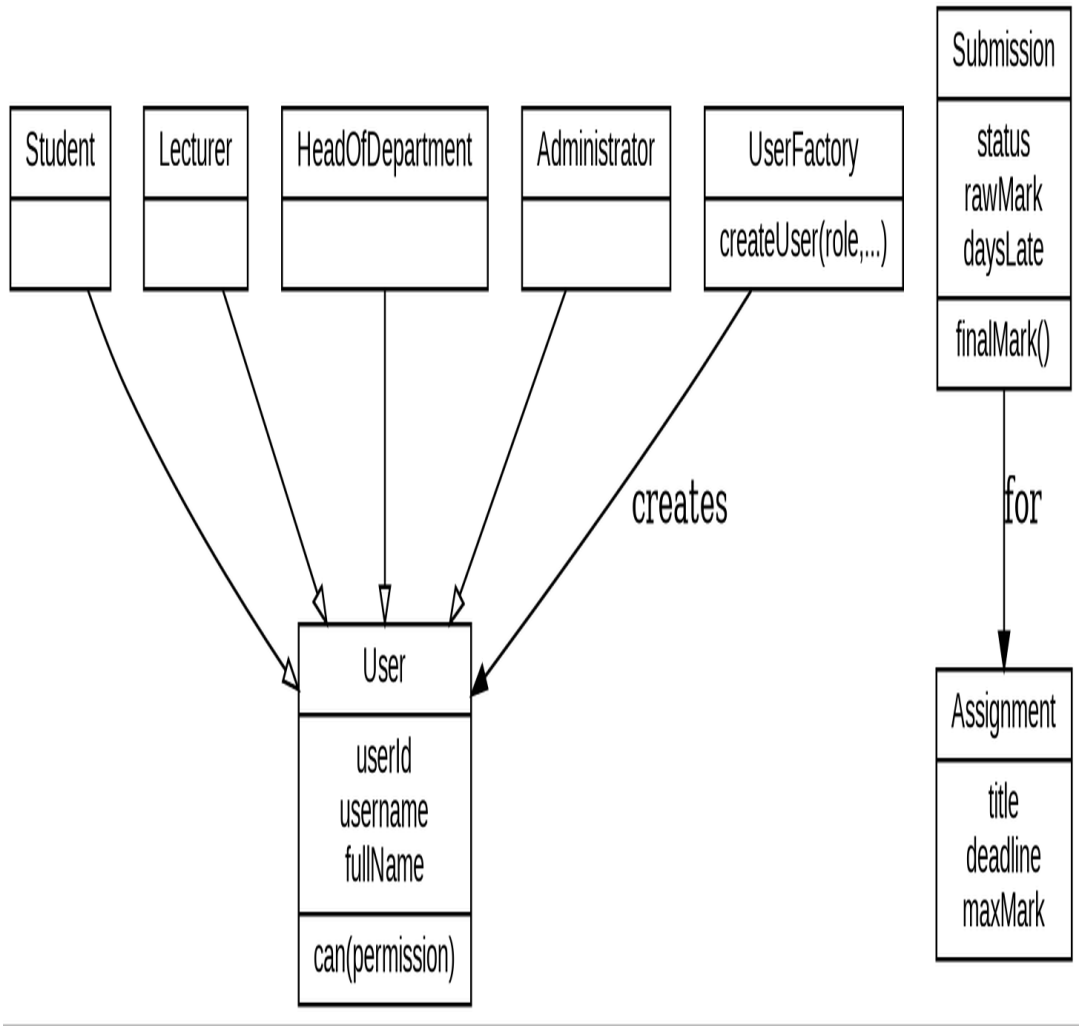
## Design rationale

| Driver | How layering satisfies it |
|---|---|
| Must work offline on old hardware | Runs as one application with a local SQLite file. No separate server is needed. |
| Role-based access for four users | Authorisation is enforced centrally in the service layer, so a bypassed menu cannot bypass the rules. |
| Rules will change | Business rules are kept in the Domain and Service layer. |
| Security: hashed passwords, parameterised SQL | SQL is kept in one layer, making it easier to check queries for injection risks. |
| Simple for all users | A simple console menu for each role, with the main work handled by the service layer. |

## Rejected alternative: Client server

A client server design would support users working over a network, but the internet is unreliable and the computers are old. A separate database server would also need to be installed and maintained, and could become a single point of failure. The system mainly handles reading, with fewer updates, so an embedded SQLite database should be suitable for the expected 800 users. The main limitation is that many users cannot write to the database at the same time over a LAN. If the system later needs a central server, the repository interfaces will make it easier to replace SQLite.

## 2. Class Diagram




### Design patterns applied

| Pattern | Where | Why chosen |
|---|---|---|
| Strategy | Late Penalty Rule | HoD's penalty rule is likely to change. Swapping a policy needs no change to Submission or its services. |
| Repository | Submission repository and SQLite repository | Hides SQL from the domain, which makes parameterised queries and the data access layer only requirement enforceable. |
| Factory | Creating a user and role. | The Administrator creates users of four roles. A factory centralises construction, password hashing. |

### 2 SOLID

S (Single responsibility): Each class has one main job. Submission manages the submission status, SqliteSubmissionRepository saves and retrieves submissions, and AuditLogger records audit entries.

O (Open/Closed): New penalty rules, observers or user roles can be added through new classes instead of changing existing code.

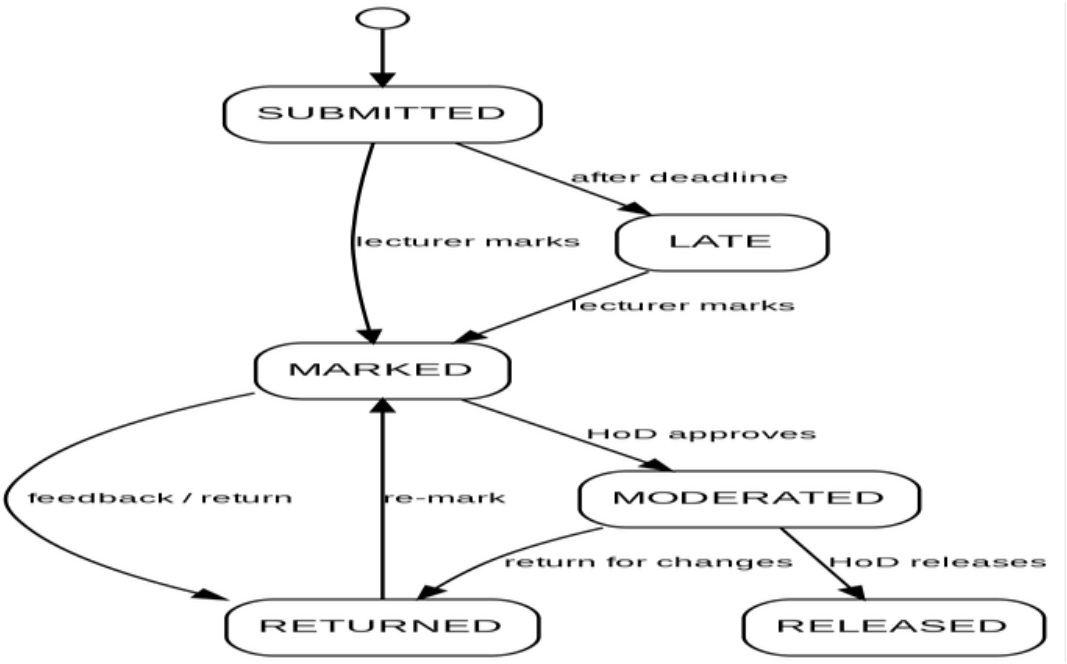
L (Liskov substitution): Student, Lecturer, HoD and Administrator are all types of users. Services can use them as users and check permissions.

I (Interface segregation): The design uses smaller interfaces such as IPasswordHasher, ILatePenaltyPolicy and ISubmissionObserver instead of one large interface. Each repository also exposes only the operations its clients need.

D (Dependency inversion): Services use interfaces such as IRepository, ILatePenaltyPolicy and IPasswordHasher rather than depending directly on specific implementations. The SQLite repositories and password hashing class are connected.

## 3. Behavioural Models

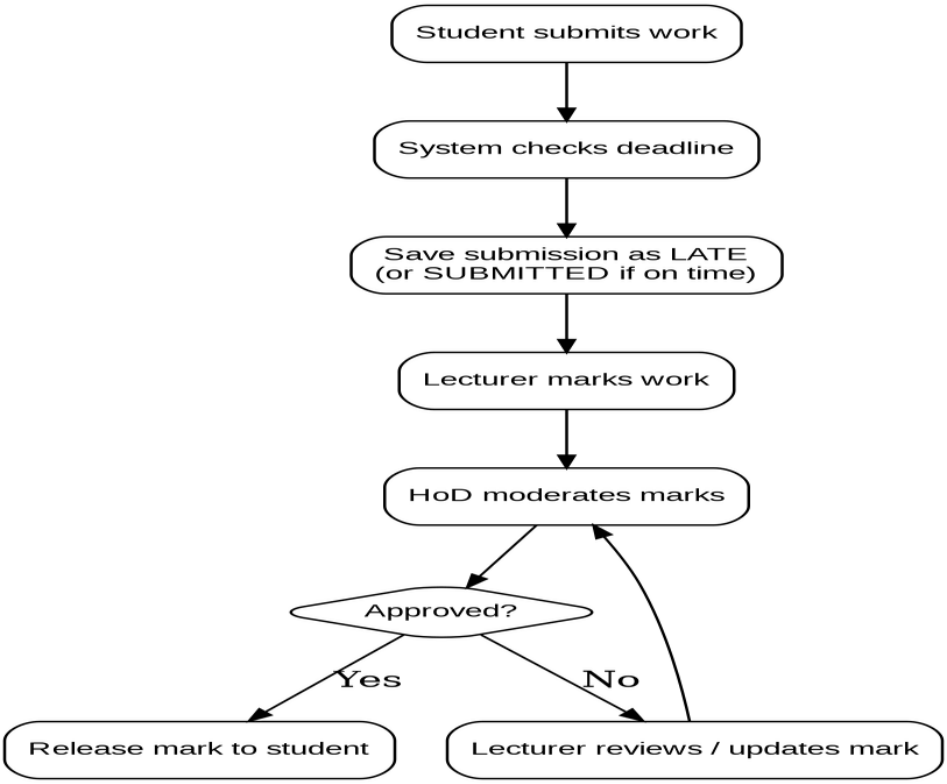
### Submission life cycle



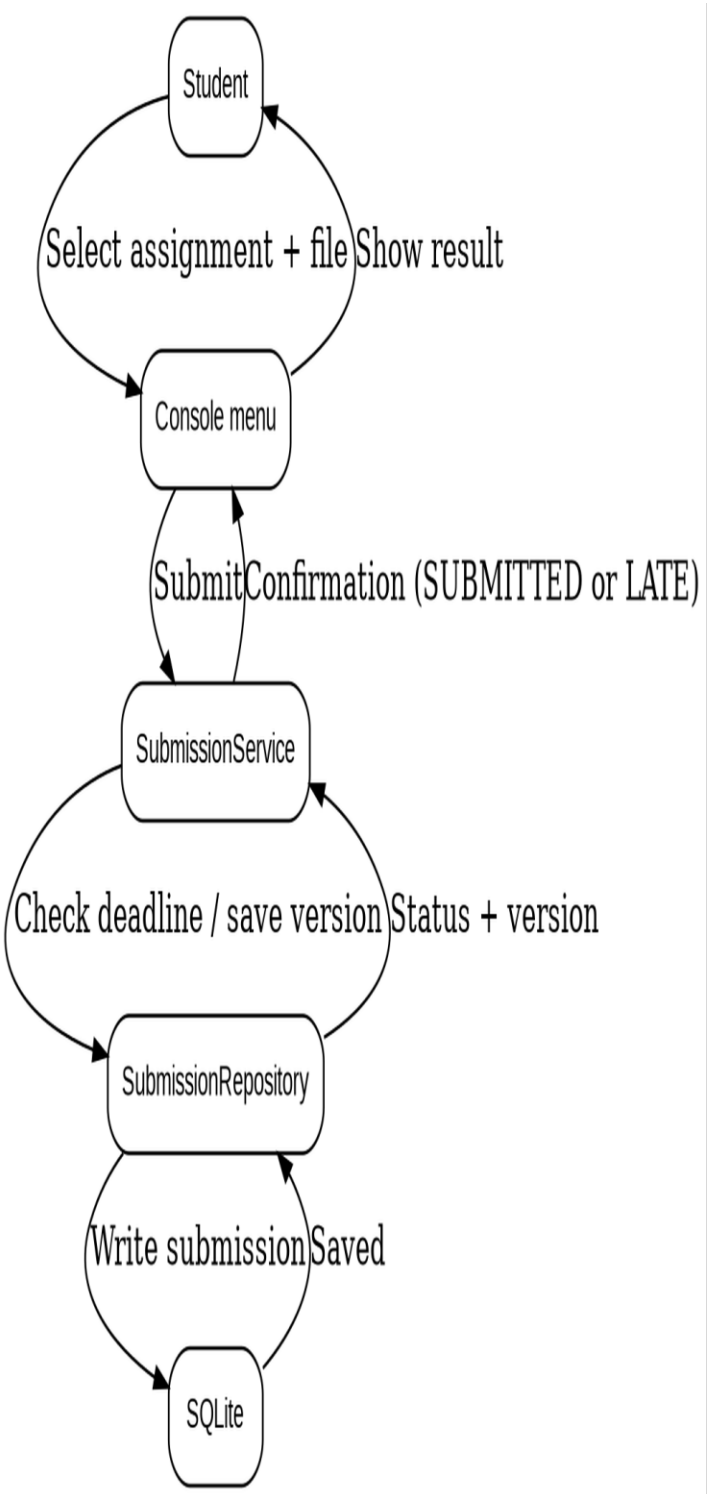

The main rules for the state machine are:

- A student can resubmit only while the effective deadline has not passed.
- A mark can be released only after moderation. This prevents students from seeing marks before the HoD releases them.
- Whenever a mark changes, a mark audit record stores who changed it, when it changed, the old and new values, and the reason.
- If the HoD changes the mark during moderation, the submission remains in MODERATED. A reason for the change must be recorded.

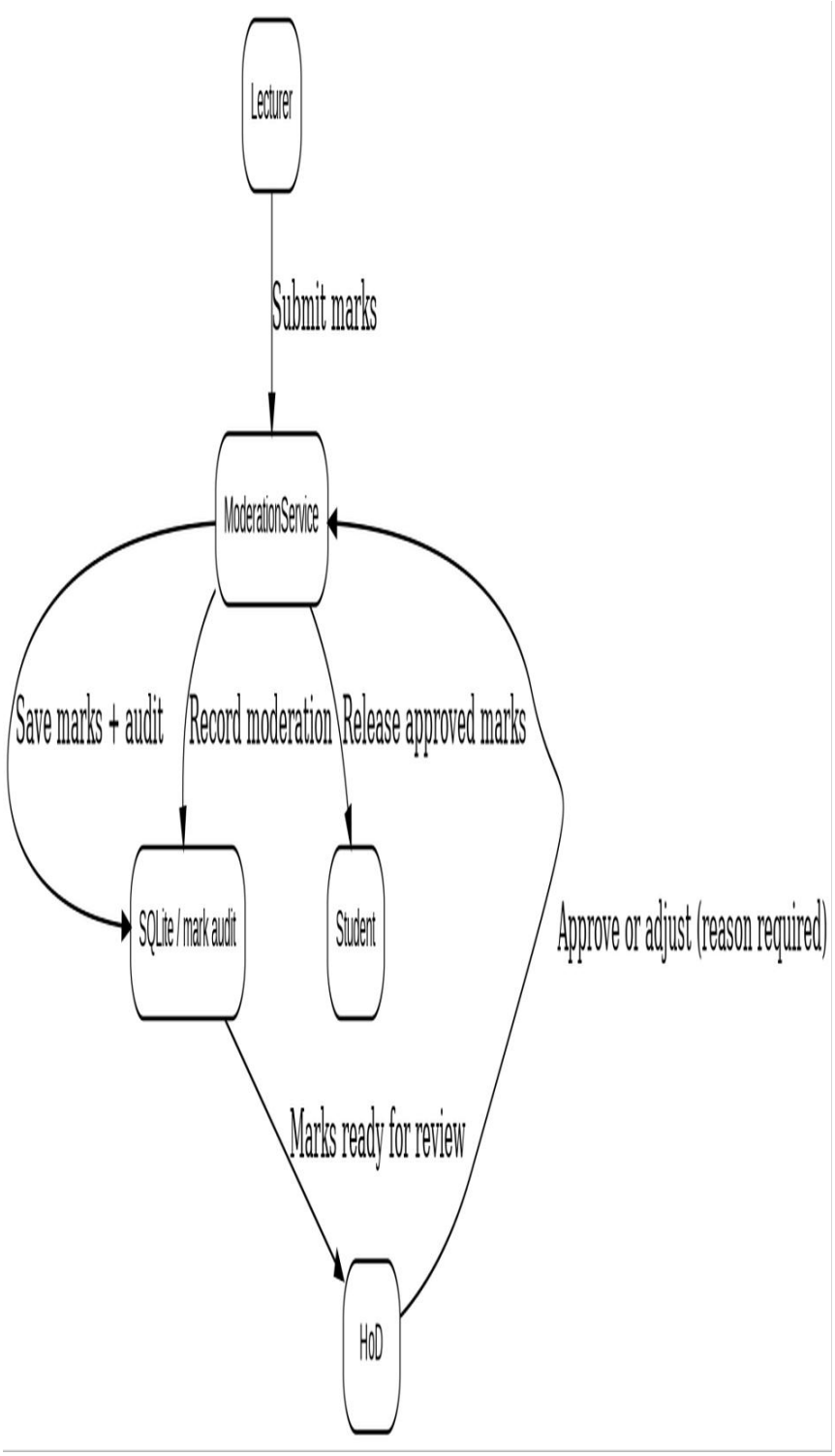
Activity Diagram with swimlanes



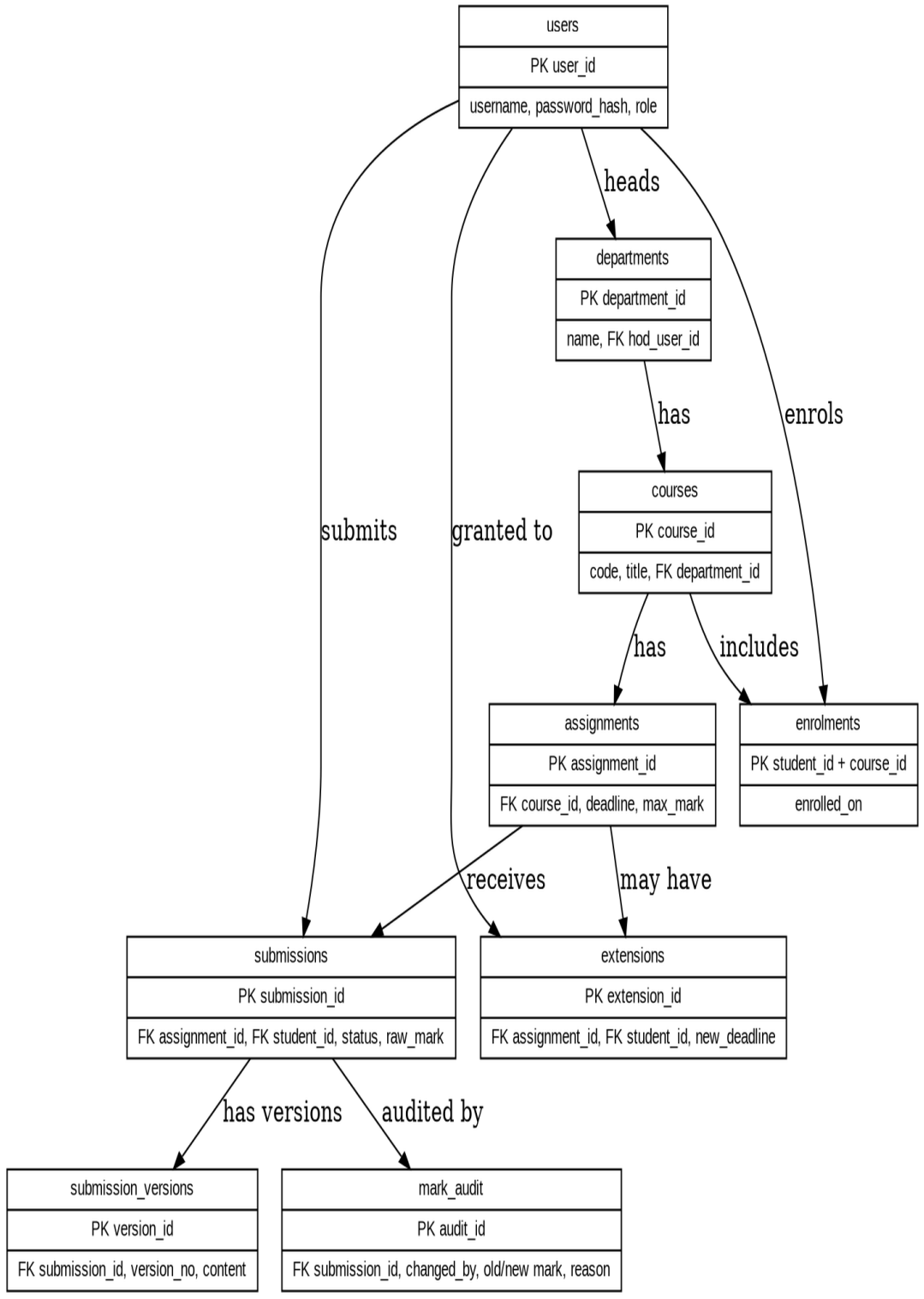
Sequence Diagram: Submit assignment



Sequence Diagram: Moderate and release marks



## 4. Database Design



## Normalisation

1NF: Each field stores one value. Resubmissions are stored in submission_versions instead of repeating columns in one row. The enrolments and course_lecturers tables handle many-to-many relationships.

2NF: In tables with a composite primary key, such as enrolments and course_lecturers, each non-key field depends on the whole key.

3NF: The final mark is calculated from raw_mark, days_late and assignments.max_mark, so it is calculated when needed instead of being stored twice. ILatePenaltyPolicy applies the rule. Submissions store lecturer and HoD IDs rather than repeating their names.

## 4 Data dictionary

| Table | Column | Type | Constraint | Description |
|---|---|---|---|---|
| users | user_id | INTEGER | | Unique user id |
| users | username | TEXT | UNIQUE, NOT NULL | Login name |
| users | password_hash | TEXT | NOT NULL | Salted hash (never plain text) |
| users | role | TEXT | CHECK in (STUDENT, LECTURER, HOD, ADMIN) | Role discriminator |
| users | full_name | TEXT | NOT NULL | Display name |
| users | is_active | INTEGER | DEFAULT 1 | Soft-disable account |
| departments | department_id | INTEGER | PK | |
| departments | name | TEXT | UNIQUE, NOT NULL | |
| departments | hod_user_id | INTEGER | FK → users, UNIQUE, NULL | The HoD |
| courses | course_id / code / title / department_id | INTEGER/TEXT/TEXT/INTEGER | PK / UNIQUE / NOT NULL / FK | Course catalogue |
| course_lecturers | course_id, lecturer_id | INTEGER | composite PK, FKs | Who teaches which course |
| enrolments | student_id, course_id, enrolled_on | INTEGER, INTEGER, TEXT | composite PK, FKs | Student enrolment |
| assignments | assignment_id … created_by | – | PK; FK course; max_mark > 0 | Assignment definition |
| assignments | deadline | TEXT (ISO-8601 UTC) | NOT NULL | Base deadline |
| extensions | assignment_id, student_id | INTEGER | UNIQUE pair | One personal extension per student per assignment |
| extensions | new_deadline, reason, granted_by, granted_at | TEXT/TEXT/INTEGER/TEXT | NOT NULL | Audit of who granted what and why |
| submissions | status | TEXT | CHECK in 6 states | Life-cycle state |

| Table | Column | Type | Constraint | Description |
|---|---|---|---|---|
| submissions | raw_mark | REAL | NULL until marked; >= 0 | Mark before penalty |
| submissions | days_late | INTEGER | DEFAULT 0, >= 0 | Whole days late |

## SQL schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE users (
  user_id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  salt TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('STUDENT','LECTURER','HOD','ADMIN')),
  full_name TEXT NOT NULL,
  is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1))
);

CREATE TABLE departments (
  department_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE,
  hod_user_id INTEGER UNIQUE REFERENCES users(user_id)
);

CREATE TABLE courses (
  course_id INTEGER PRIMARY KEY AUTOINCREMENT,
  code TEXT NOT NULL UNIQUE,
  title TEXT NOT NULL,
  department_id INTEGER NOT NULL REFERENCES departments(department_id)
);

CREATE TABLE course_lecturers (
  course_id INTEGER NOT NULL REFERENCES courses(course_id) ON DELETE CASCADE,
  lecturer_id INTEGER NOT NULL REFERENCES users(user_id),
  PRIMARY KEY (course_id, lecturer_id)
);

CREATE TABLE enrolments (
  student_id INTEGER NOT NULL REFERENCES users(user_id),
  course_id INTEGER NOT NULL REFERENCES courses(course_id) ON DELETE CASCADE,
  enrolled_on TEXT NOT NULL DEFAULT (datetime('now')),
  PRIMARY KEY (student_id, course_id)
);

CREATE TABLE assignments (
  assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_id INTEGER NOT NULL REFERENCES courses(course_id),
  title TEXT NOT NULL,
  description TEXT,
  deadline TEXT NOT NULL,
  max_mark REAL NOT NULL CHECK (max_mark > 0),
  created_by INTEGER NOT NULL REFERENCES users(user_id)
);

CREATE TABLE extensions (
  extension_id INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id INTEGER NOT NULL REFERENCES assignments(assignment_id),
  student_id INTEGER NOT NULL REFERENCES users(user_id),
```

```
  new_deadline TEXT NOT NULL,
  reason TEXT NOT NULL,
  granted_by INTEGER NOT NULL REFERENCES users(user_id),
  granted_at TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submissions (
  submission_id INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id INTEGER NOT NULL REFERENCES assignments(assignment_id),
  student_id INTEGER NOT NULL REFERENCES users(user_id),
  status TEXT NOT NULL CHECK (status IN
  ('SUBMITTED','LATE','MARKED','RETURNED','MODERATED','RELEASED')),
  raw_mark REAL CHECK (raw_mark IS NULL OR raw_mark >= 0),
  days_late INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
  feedback TEXT,
  marked_by INTEGER REFERENCES users(user_id),
  moderated_by INTEGER REFERENCES users(user_id),
  released_at TEXT,
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_versions (
  version_id INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id INTEGER NOT NULL REFERENCES submissions(submission_id) ON DELETE CASCADE,
  version_no INTEGER NOT NULL CHECK (version_no >= 1),
  content TEXT NOT NULL,
  submitted_at TEXT NOT NULL,
  UNIQUE (submission_id, version_no)
);

CREATE TABLE mark_audit (
  audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id INTEGER NOT NULL REFERENCES submissions(submission_id),
  changed_by INTEGER NOT NULL REFERENCES users(user_id),
  changed_at TEXT NOT NULL DEFAULT (datetime('now')),
  old_value REAL,
  new_value REAL NOT NULL,
  reason TEXT NOT NULL CHECK (length(trim(reason)) > 0)
);

-- Indexes for the 800-student performance requirement
CREATE INDEX idx_sub_assignment ON submissions(assignment_id);
CREATE INDEX idx_sub_student ON submissions(student_id, status);
CREATE INDEX idx_audit_sub ON mark_audit(submission_id);
CREATE INDEX idx_enrol_course ON enrolments(course_id);
```

## Mapping tables

| Class | Table | Difference and justification |
|---|---|---|
| User and the 4 subclasses | users (single table with role) | The subclasses add almost no extra attributes, so separate tables would add joins for no benefit. The UserFactory and repository map role to the right subclass. The HoD's department is held in departments.hod_user_id, so the departmentId field in HeadOfDepartment is loaded from there. |

| Class | Table | Difference and justification |
|---|---|---|
| Department, Course, Assignment, Extension, MarkAudit | same-named tables | Direct 1:1 mapping. |
| Enrolment | enrolments | Association class from the many-to-many Student–Course relationship. |
| Course–Lecturer association | course_lecturers | Join table resolving a many-to-many relationship. |
| Submission + SubmissionVersion | submissions + submission_versions | The aggregate maps to two tables so that resubmissions are kept as history. |
| Submission::finalMark() | | A derived value that is deliberately not stored, to satisfy 3NF and to keep results consistent when the penalty policy changes. |
| ILatePenaltyPolicy, IRepository, IPasswordHasher, services, observers | | Behavioural and infrastructure classes with no persistent state. |
| Permission / can() | | Permissions are fixed per role in code. A permissions table would be unnecessary complexity for four static roles. |

# Phase 5: Verification and Validation

## 5. Test strategy and test plan

| Level | Artefact verified | Test basis | Exit criteria |
|---|---|---|---|
| Unit | Individual classes: Submission, ILatePenaltyPolicy implementations, User subclasses and can(), UserFactory, password hasher, deadline resolution, audit logger | Detailed design (class diagram, state machine, business rules in the design assumptions) | All unit tests pass. |
| Integration | Services working with SQLite repositories through the repository. | Architecture, repository pattern, SQL schema and mapping tables | All integration tests pass. |
| System | The complete application driven through the console menus for all four roles | SRS and use case. | Every Must requirement has at least one executed system test; zero open Critical or High defects; Medium defects have an agreed workaround |
| Acceptance | The delivered system against what the five stakeholders asked for and the conflict resolutions in the design assumptions | Stakeholder statements and conflict-resolution table | Every acceptance test passed. |

| Partition | Range of daysLate | Valid? | Expected behaviour |
|---|---|---|---|
| P1: on time | 0 | Valid | Final mark = raw mark |
| P2: penalised | 1, 2, 3 | Valid | Final mark = max(0, raw − 0.10 × maxMark × daysLate) |
| P3: zero band | 4 and above | Valid | Final mark = 0 |
| P4: impossible negative | Negative | Invalid | Rejected with an exception (std::invalid_argument) |

## Validation

| Stakeholder statement | Acceptance check | Pass criterion | Result |
|---|---|---|---|
| Principal: students must not see marks before the HoD has checked them | As a student, look for the mark at every status before RELEASED | No mark or partial mark is visible until RELEASED | Pass |
| Lecturer: sees marks as soon as marked; may grant extensions | Lecturer marks work and views it immediately; grants an extension | Lecturer sees the mark at once; extension changes the student's effective deadline | Pass |
| HoD: 10% per day for up to three days, then zero; sees every mark before release | Run UT-02 to UT-05 values through the full application; HoD views the department's marks in bulk | Final marks 70, 60, 50, 0 for raw 80/100; HoD list complete for own department only | Pass |
| Student representative: resubmit before deadline; written feedback; late work not rejected | ST-05, ST-07 and a feedback check | Resubmission works; late work accepted and flagged; feedback visible after release | pass |
| IT officer: works offline | Disconnect the network; run all four role sessions | All functions work with no network | pass |

# 6 Implementation

The implementation translates the requirements and design into a role-based C++ application backed by an SQLite database. The system is intended to operate locally so that the school can continue using it without an internet connection. The application is organised into presentation, service, domain and data-access responsibilities. The presentation layer provides the menus and collects user input. The service layer coordinates activities such as account management, assignment creation, submission, marking and moderation. Domain classes represent the main concepts and enforce rules that should not depend on a particular screen or database query. The data-access layer is responsible for communicating with SQLite

# Phase 7: Deployment

The system is a single-process C++17 application with an embedded SQLite database, as chosen in the architecture (offline, old hardware, no database server). It is installed on one school computer. Other people use it through a terminal session on that computer, directly or by remote terminal over the school LAN. Staff PCs never open the database file themselves.




**Figure 7.1** Deployment diagram for release

## 7 Installation guide

### Installation steps

1. Log in to the school computer as an administrator account of the operating system.
2. Copy the release folder `lms-1.0` (from the USB drive or the Git release tagged `v1.0`) to `C:\LMS` on Windows or `/opt/lms` on Ubuntu.
3. Check the folder contains: `lms` (the program), `schema.sql`, `README.txt` and the `manuals` folder.
4. Create a data folder: `C:\LMS\data` or `/opt/lms/data`. Give only the school's LMS operating-system account permission to read and write it, because the database holds learners' marks.
5. Open a terminal in the release folder and run `lms --init`. The program creates `data/lms.db` from `schema.sql` and asks you to create the first Administrator account. Choose a strong password and record it in the school's secure password record.
6. Run `lms`, log in as the Administrator and create one test user of each role. Confirm each role sees only its own menu.
7. Schedule a daily backup (see below) and test one restore before handing over.
8. Give each role its user manual (section 7.3).

## User manuals

### Administrator manual

1. **Log in.** Enter your username and password. Choose Administrator menu.
2. **Create a user.** Choose Manage users, then Add user. Enter full name, a unique username, the role (Student, Lecturer, Head of Department or Administrator) and an initial password. Tell the user to keep it private.
3. **Create a department and assign its Head.** Departments → Add → choose the HoD (one HoD per department).
4. **Create a course.** Courses,Add. Enter code, title and department.
5. **Assign lecturers and enrol students.** Open the course, choose Assign lecturer or Enrol student.

6. **Deactivate an account** for someone who has left. The account's history is kept; it can no longer log in.

## Lecturer manual

1. **Create an assignment.** Choose your course, then New assignment. Enter title, description, deadline and maximum mark.
2. **Grant an extension.** Open the assignment, choose Grant extension, pick the student, enter a later date and the reason. The reason is required and is stored with your name. The student's penalty is then measured from the new date.
3. **Mark work.** Open Submissions, choose a student, read the latest version, enter the mark (0 to the maximum) and written feedback. You will see your marks immediately. Students will not see them yet.
4. **Late work.** Late submissions are marked as normal and show LATE and days late. The system applies the late penalty automatically; you do not calculate it.
5. **Changing a mark later.** You must enter a reason. The old and new marks are recorded.

## Head of Department manual

1. **Review.** Choose Moderate marks. The list shows all marked submissions for your department's courses.
2. **Approve.** Select submissions (or approve all in a course in bulk). Status becomes MODERATED.
3. **Adjust a mark.** Choose Adjust, enter the new mark and the reason. The reason is required.
4. **Return for changes.** Choose Return and the lecturer re-marks the work.
5. **Release.** Choose Release for moderated submissions. Only then can students see marks and feedback.

## Student manual

1. **Log in** and choose My courses to see enrolled courses and assignments with deadlines.
2. **Submit.** Choose the assignment, enter the file path, and confirm.
3. **Resubmit.** You may submit again before the deadline. Every version is kept and the latest is marked. After the deadline, resubmission is not allowed.
4. **Late work.** It is still accepted, but marks are reduced by 10% of the maximum mark for each day late, up to three days; after that the mark is 0. Ask your lecturer for an extension before the deadline if you have a good reason.
5. **See results.** Choose My results. Marks and feedback appear only after the Head of Department has released them.

# Phase 8: Maintenance: Change Request CR1

**CR1:** "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."

## Classification and justification

CR1 is a maintenance upgrade. Nothing has to be corrected because the system functions as intended. It is not adaptable because the operating environment has not changed. Restructuring is not preventive because it is not meant to stop a future error. A visible extension on the student's list and a whole-class scope for extensions are two new or improved behaviors that the school is requesting. The impact study that follows covers all of the requirements and tests necessary for improvements.

## Impact analysis

| Artefact type | Item | Affected? | Change needed |
|---|---|---|---|
| Requirements | Lecturer grants extension to an individual student | Modified | Widen to individual or whole class |
| | Student views assignments and deadlines | Modified | Show extension and effective deadline |
| | Resubmission only before the deadline | Clarified | "Deadline" means effective deadline including class extension |
| | Late penalty rule | Clarified | Measured from the effective deadline; no change to the formula |
| | New requirements | Added | CR1-R1: class extension; CR1-R2: extension shown on assignment list (add to SRS, MoSCoW: Should) |
| Use cases | Grant extension | Modified | Alternative flow: lecturer chooses "whole class"; new exception: date not later than base |
| | Submit assignment | Modified | Main flow step "check deadline" uses the effective deadline |
| | View assignments / courses (student) | Modified | Main flow shows extension label |
| Diagrams | Class diagram | Modified | Add ClassExtension; add DeadlineResolver; new methods on repository interface |
| | Sequence: Submit assignment | Modified | Insert call to DeadlineResolver before the deadline check |

| Artefact type | Item | Affected? | Change needed |
|---|---|---|---|
| Classes | Assignment | Modified | effectiveDeadlineFor(studentId) delegates to DeadlineResolver |
| | DeadlineResolver (new) | Added | Computes the latest of base, class and personal deadlines |
| | ClassExtension (new entity) | Added | assignmentId, newDeadline, reason, grantedBy, grantedAt |
| Tables | class_extensions | Added | New table (script below) |

## Regression testing

| Test | Expected |
|---|---|
| No extension at all | Effective deadline = base |
| Class extension only | Effective deadline = class date for every enrolled student |
| Personal extension later than class extension | Personal date wins (the latest) |
| Personal extension earlier than class extension | Class date wins; the extension never shortens |
| Class extension date equal to or earlier than base | Rejected |
| Class extension with empty reason | Rejected |
| Student submits after the base deadline but before the class date | SUBMITTED, not LATE |
| Student submits 1 second after the class date | LATE, daysLate = 1, penalty from the class date |
| Student resubmits between base deadline and class date | Allowed; new version stored |
| Student enrolled after the extension was granted | Gets the class extension |

# Phase 9: Professional, Legal and Ethical Issues

## Data protection personal information

Lesotho's Data Protection Act, 2011 (Act No. 5 of 2012, published in the Gazette on 22 February 2012) governs how personal information is collected, processed and stored, and it places duties on the data controller, which in this case is the school. Personal Information of the system's users is protected by password and the database is with the school, therefore information shall always be protected and a possible leak of it is solely the school's responsibility.

## Professional and ethical responsibilities of the developer

Code of ethics. The ACM/IEEE-CS Software Engineering Code of Ethics asks for public interest, honesty about limits and competence. In practice: this report states the known limits (no encryption at rest, console-only, single-writer database) instead of hiding them, and releases are tested before deployment.

Honest reporting of testing. Coverage figures and defect logs are reported as measured, including defects found.

Duty of care after release. Change requests such as CR1 go through impact analysis and regression testing so that a convenience feature cannot weaken the integrity of marks.

Scope honesty. Plagiarism detection and automated grading are out of scope; the system does not judge academic honesty, so staff must not treat it as doing so.

# References


- Government of Lesotho (2012). Data Protection Act, 2011 (Act No. 5 of 2012). Lesotho Government Gazette, Vol. 57, No. 19, 22 February 2012. Available at lesotholii.org.
- ACM and IEEE Computer Society (1999). Software Engineering Code of Ethics and Professional Practice.
- W3C (2018). Web Content Accessibility Guidelines (WCAG) 2.1, used here as a principle checklist, not as a legal standard.